BFS

Level to find goal:

```
from collections import deque


def bfs(grid, start, goal):
    N = len(grid)
    moves = [(1,0), (-1,0), (0,1), (0,-1)]
    q = deque([(start[0], start[1], 0)])  # (x, y, level)
    grid[start[0]][start[1]] = 0  # mark visited

    while q:
        x, y, level = q.popleft()
        if (x, y) == goal:
            return level  # goal found

        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < N and 0 <= ny < N and grid[nx][ny] == 1:
                grid[nx][ny] = 0
                q.append((nx, ny, level + 1))

    return -1  # goal not reachable

# Example usage
grid = [
    [0, 0, 1, 0, 1],
```

```
    [0, 1, 1, 1, 1],
    [0, 1, 0, 0, 1],
    [1, 1, 0, 1, 1],
    [1, 0, 0, 0, 1]
]

start = (0, 2)
goal = (4, 4)
steps = bfs(grid, start, goal)

if steps != -1:
    print("Goal found in", steps, "moves")
else:
    print("Goal cannot be reached")
```

path to find goal:

```
from collections import deque

def bfs(grid, start, goal):
    N = len(grid)
    moves = [(1,0), (-1,0), (0,-1), (0,1)]
    q = deque([(start[0], start[1], [start])])  # store path
    grid[start[0]][start[1]] = 0  # mark visited

    while q:
        x, y, path = q.popleft()
```

```
        if (x, y) == goal:
            return path  # return full path when goal found

        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < N and 0 <= ny < N and grid[nx][ny] == 1:
                grid[nx][ny] = 0
                q.append((nx, ny, path + [(nx, ny)]))



    return None  # goal not reachable

grid = [
    [0, 0, 1, 0, 1],
    [0, 1, 1, 1, 1],
    [0, 1, 0, 0, 1],
    [1, 1, 0, 1, 1],
    [1, 0, 0, 0, 1]
]

start = (0, 2)
goal = (4, 4)
path = bfs(grid, start, goal)

if path:
    print("Goal found!")
    print("Number of moves:", len(path) - 1)
```

```
    print("Path:")
    for step in path:
        print(step)
else:
    print("Goal cannot be reached")
```

DFS

Iterative:

```
def dfs_iterative(grid, start, goal):
    N = len(grid)
    moves = [(1,0), (-1,0), (0,1), (0,-1)]
    stack = [(start[0], start[1], [start], 0)]  # (x, y, path, depth)
    grid[start[0]][start[1]] = 0  # mark start visited

    while stack:
        x, y, path, depth = stack.pop()

        if (x, y) == goal:
            return path, depth

        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < N and 0 <= ny < N and grid[nx][ny] == 1:
                grid[nx][ny] = 0
                stack.append((nx, ny, path + [(nx, ny)], depth + 1))
```

```
    return None, None  # goal not reachable



# Example grid
grid = [
    [0, 0, 1, 0, 1],
    [0, 1, 1, 1, 1],
    [0, 1, 0, 0, 1],
    [1, 1, 0, 1, 1],
    [1, 0, 0, 0, 1]
]

start = (0, 2)
goal = (4, 4)
path, depth = dfs_iterative(grid, start, goal)

if path:
    print("Goal found!")
    print("Total depth:", depth)
    print("Path:")
    for step in path:
        print(step)
else:
    print("Goal cannot be reached")
```

Recursive

```
def dfs(grid, current, goal, path, depth):
    x, y = current  # unpack current position

    # Goal check
    if current == goal:
        return True, path, depth

    # Mark visited
    grid[x][y] = 0
    moves = [(1,0), (-1,0), (0,1), (0,-1)]
    N = len(grid)

    # Explore all directions
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < N and 0 <= ny < N and grid[nx][ny] == 1:
            found, final_path, final_depth = dfs(grid, (nx, ny), goal, path + [(nx, ny)], depth + 1)
            if found:
                return True, final_path, final_depth

    return False, path, depth



# Example grid
grid = [
    [0, 0, 1, 0, 1],
```

```
    [0, 1, 1, 1, 1],
    [0, 1, 0, 0, 1],
    [1, 1, 0, 1, 1],
    [1, 0, 0, 0, 1]
]

start = (0, 2)
goal = (4, 4)

found, path, depth = dfs(grid, start, goal, [start], 0)

if found:
    print("Goal found!")
    print("Total depth:", depth)
    print("Path:")
    for step in path:
        print(step)
else:
    print("Goal cannot be reached")
```

IDDFS

```
# Depth-Limited Search (Recursive helper function)
def dls(graph, current, goal, path, depth, limit, visited):
    """
    Performs Depth-Limited Search (DLS) recursively.
    Stops expanding when the given depth limit is reached.
    """
    # Goal check
    if current == goal:
        return True, path, depth

    # Stop recursion if the limit is reached
    if depth >= limit:
        return False, path, depth

    # Explore all neighbors of current node
    for neighbor in range(len(graph[current])):
        if graph[current][neighbor] == 1 and neighbor not in visited:
            visited.add(neighbor)
            found, final_path, final_depth = dls(
                graph, neighbor, goal, path + [neighbor], depth + 1,
limit, visited
            )
```

```
            if found:
                return True, final_path, final_depth

            # Backtrack
            visited.remove(neighbor)

    # Goal not found in this branch
    return False, path, depth



# Main IDDFS Function
def iddfs(graph, start, goal, max_depth=20):
    """
    Iterative Deepening DFS — Repeatedly calls DLS with increasing depth
limit.
    Returns the path and final depth when goal is found.
    """
    for limit in range(max_depth + 1):
        print(f" Searching at depth limit: {limit}")
        visited = {start}

        found, path, depth = dls(graph, start, goal, [start], 0, limit,
visited)

        if found:
            print(f"\n Goal found at depth {depth} (limit {limit})")
            return path, depth
```

```
    print("\n Goal not found within max depth limit.")
    return None, None


print("The number of nodes in the graph is 7")

graph = [
    [0, 1, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 1, 1],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0]
]

start = 0   # Node 1 (index 0)
goal = 6    # Node 7 (index 6)
print("The destination for the graph is 7")


path, depth = iddfs(graph, start, goal, max_depth=10)


if path:
```

```
    print("\n Path to goal:")
    for node in path:
        print("Node", node + 1)
    print(" Total depth:", depth)
else:
    print("Goal cannot be reached.")
```

Graph Coloring

```
def is_safe(graph, node, color, c):
    for i in range(len(graph)):
        if graph[node][i] == 1 and color[i] == c:
            return False
    return True

def graph_coloring(graph, m, color, node=0):
    if node == len(graph):
        return True

    for c in range(m):
        if is_safe(graph, node, color, c):
            color[node] = c
            if graph_coloring(graph, m, color, node + 1):
                return True
            color[node] = -1  # backtrack
    return False

# --- main part ---
graph = [
    [0, 1, 1, 1],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [1, 0, 1, 0]
```

```
]

m = 3  # number of colors
color = [-1] * len(graph)
color_names = [ "RED", "GREEN", "BLUE", "YELLOW", "ORANGE", "PINK"]

if graph_coloring(graph, m, color):
    print("Solution exists!\n")
    for i in range(len(graph)):
        print(f"Node {i + 1} = {color_names[color[i]]}")
    used_colors = set(color)
    print(f"\nNumber of colors used: {len(used_colors)}")

else:
    print("No solution exists.")
```

N Queen Problem

```
def print_board(board, N):
    for i in range(N):
        for j in range(N):
            print("Q" if board[i][j] else ".", end=" ")
        print()
    print()


def is_safe(board, row, col, N):
    # Check this row on left side
    for i in range(col):
        if board[row][i]:
            return False
```

```
    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j]:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j]:
            return False

    return True



def solve_nq_util(board, col, N):
    if col >= N:
        print_board(board, N)
        return True

    res = False
    for i in range(N):
        if is_safe(board, i, col, N):
            board[i][col] = 1
            res = solve_nq_util(board, col + 1, N) or res
            board[i][col] = 0  # backtrack
    return res
```

```python
def solve_nq(N):
    board = [[0] * N for _ in range(N)]
    if not solve_nq_util(board, 0, N):
        print("No solution exists")
    else:
        print("Solutions above")



# Example
solve_nq(6)
```

Minmax

```python
import math

def minimax(curDepth, nodeIndex, maxTurn, scores, targetDepth):
# Base case: reached leaf node
 if curDepth == targetDepth:
  return scores[nodeIndex]

 if maxTurn:
    return max(
        minimax(curDepth + 1, nodeIndex*2, False, scores, targetDepth),
```

```
        minimax(curDepth + 1, nodeIndex*2 + 1, False, scores, targetDepth)
    )
 else:
    return min(
        minimax(curDepth + 1, nodeIndex*2, True, scores, targetDepth),
        minimax(curDepth + 1, nodeIndex*2 + 1, True, scores, targetDepth)
    )


n = int(input("Enter number of leaf nodes (power of 2): "))
scores = [int(input(f"Score for leaf {i}: ")) for i in range(n)]
treeDepth = int(math.log2(n))

print("The optimal value is:", minimax(0, 0, True, scores, treeDepth))
```

alpha beta

```
import math

def alphabeta(curDepth, nodeIndex, isMax, scores, targetDepth, alpha,
beta):
    # Base case: leaf node
    if curDepth == targetDepth:
        return scores[nodeIndex]
```

```
if isMax:
    best = -999999

    # Left child
    val = alphabeta(curDepth + 1, nodeIndex * 2, False,
                    scores, targetDepth, alpha, beta)
    best = max(best, val)
    alpha = max(alpha, best)

    # Pruning check
    if beta <= alpha:
        return best

    # Right child
    val = alphabeta(curDepth + 1, nodeIndex * 2 + 1, False,
                    scores, targetDepth, alpha, beta)
    best = max(best, val)
    alpha = max(alpha, best)

    return best

else:  # Min player
    best = 999999

    # Left child
    val = alphabeta(curDepth + 1, nodeIndex * 2, True,
```

```
                    scores, targetDepth, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)

        # Pruning check
        if beta <= alpha:
            return best

        # Right child
        val = alphabeta(curDepth + 1, nodeIndex * 2 + 1, True,
                    scores, targetDepth, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)

        return best


# ---------- MAIN PROGRAM ----------
n = int(input("Enter number of leaf nodes (power of 2): "))
scores = [int(input(f"Score for leaf {i}: ")) for i in range(n)]
treeDepth = int(math.log2(n))

alpha = -999999
beta = 999999

print("The optimal value is:",
      alphabeta(0, 0, True, scores, treeDepth, alpha, beta))
```

Fuzzy Rule

```
!pip install scikit-fuzzy


import numpy as np

import skfuzzy as fuzz

from skfuzzy import control as ctrl


# Define fuzzy variables

temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')

humidity    = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')

power       = ctrl.Consequent(np.arange(0, 101, 1), 'power')


# Membership functions for Temperature

temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 20])

temperature['warm'] = fuzz.trimf(temperature.universe, [15, 25, 35])

temperature['hot']  = fuzz.trimf(temperature.universe, [30, 40, 40])


# Membership functions for Humidity

humidity['dry']         = fuzz.trimf(humidity.universe, [0, 0, 40])

humidity['comfortable'] = fuzz.trimf(humidity.universe, [30, 50, 70])

humidity['humid']       = fuzz.trimf(humidity.universe, [60, 100, 100])


# Membership functions for Power
```

```
power['low']    = fuzz.trimf(power.universe, [0, 0, 50])
power['medium'] = fuzz.trimf(power.universe, [30, 50, 70])
power['high']   = fuzz.trimf(power.universe, [60, 100, 100])

# Rules
rule1 = ctrl.Rule(temperature['cold'] & humidity['dry'],
power['low'])
rule2 = ctrl.Rule(temperature['warm'] & humidity['comfortable'],
power['medium'])
rule3 = ctrl.Rule(temperature['hot']  | humidity['humid'],
power['high'])
rule4 = ctrl.Rule(temperature['warm'] & humidity['humid'],
power['high'])
rule5 = ctrl.Rule(temperature['cold'] & humidity['humid'],
power['medium'])

# Control system
comfort_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5])
comfort = ctrl.ControlSystemSimulation(comfort_ctrl)

# ----- Input values (store in variables) -----
temp_val = 29
hum_val = 65

comfort.input['temperature'] = temp_val
comfort.input['humidity'] = hum_val

# Run fuzzy inference
comfort.compute()
```

```python
# Print using our own variables + output from system
print(f"Temperature: {temp_val} °C")
print(f"Humidity: {hum_val} %")
print(f"Recommended Power Level: {comfort.output['power']:.2f} %")

# Visualize result
power.view(sim=comfort)
```